JobPing — Knowledge Transfer Guide

Oct 3, 2026 · @Ankit

1. What JobPing is (the 30-second answer)

JobPing is a job-alert website for software developers. It collects new jobs from free public job sites every 2 hours, compares each job against the skills you told it you have, and emails you when a job matches well enough.

If someone asks "what does it do?", say: "You sign up, pick your skills and preferred locations, and set a minimum match percentage. A background worker pulls jobs from five job APIs every two hours, scores every new job against your skills, and if the score is above your threshold, you get an in-app notification and an email."

The user journey in 4 steps:

1. **Register** — create an account (4-step wizard: account → skills → preferences → notifications).
2. **Set preferences** — choose skills (e.g. C#, Angular, Docker), locations, and a minimum match % (default 50%).
3. **System works in the background** — jobs are fetched, scored, and matched automatically. No action from the user.
4. **Get alerted** — a notification appears in the bell icon and an email is queued for sending.

The user can also browse all jobs, filter them, save favourites, and see their "best matches" on the dashboard.

Where the code lives: GitHub repo `Ankit-gupta-work/JobPing` (public). The full original spec is `CLAUDE.md` at the repo root; the `.ai-context/` folder holds the status notes this guide is built from.

2. Tech stack — what each piece does

Eight pieces do the work. Each has one clear job; if you can explain the "job" column, you can explain the stack.

Piece	Technology	Its job in JobPing
Backend API	.NET 9 Web API (C#)	Handles every request from the website: login, jobs, preferences, alerts
Database	PostgreSQL 15 + EF Core 9 (code-first)	Permanent storage: users, jobs, skills, alerts, email queue
Cache + locks	Redis 7	Fast temporary storage: caches common reads, prevents two workers fetching at once, stores logged-out tokens
Message queue	RabbitMQ 3	Passes "new job" and "send email" messages between background workers, with retries
Email	SendGrid v3	Actually sends the alert emails (needs a real API key — not configured yet)
Frontend	Angular 21 (standalone components) + Tailwind 3 + Angular Material	The website the user clicks on
Auth	JWT (15-min access token) + refresh tokens, BCrypt password hashing	Proves who the user is on every request
Packaging	Docker Compose + nginx	Runs all 6 services together; nginx serves the website and forwards `/api` calls

Job sources (5): Remotive, Arbeitnow, We Work Remotely (RSS feed), Adzuna, and JSearch (via RapidAPI). Adzuna and JSearch need free API keys; without keys they quietly skip and return nothing.

Versions note: the project was first planned for .NET 8 / Angular 18 and was upgraded to .NET 9 / Angular 21 early on.

3. Architecture — how the code is organised

The backend uses **Clean Architecture with 4 layers**. The one rule: a controller never talks to the database or Redis directly. It always goes Controller → Service → Database/Redis.

Layer (project)	What lives here	Depends on
`JobPing.API`	Controllers (HTTP endpoints), `Program.cs` (startup + DI), middleware	Application, Infrastructure
`JobPing.Application`	Interfaces (e.g. `IJobService`) and DTOs only — no real code	Domain
`JobPing.Infrastructure`	The real implementations: EF Core DbContext, Redis, RabbitMQ, SendGrid, job API clients, background workers	Application, Domain
`JobPing.Domain`	18 plain entity classes (User, Job, Skill…)	Nothing

Why this way? Each layer can be changed without breaking the others. Example from this project: the RabbitMQ publisher started as a fake "just log it" class. Because everything used the `IMessagePublisher` interface, swapping in the real one later needed no changes anywhere else.

Key folders:

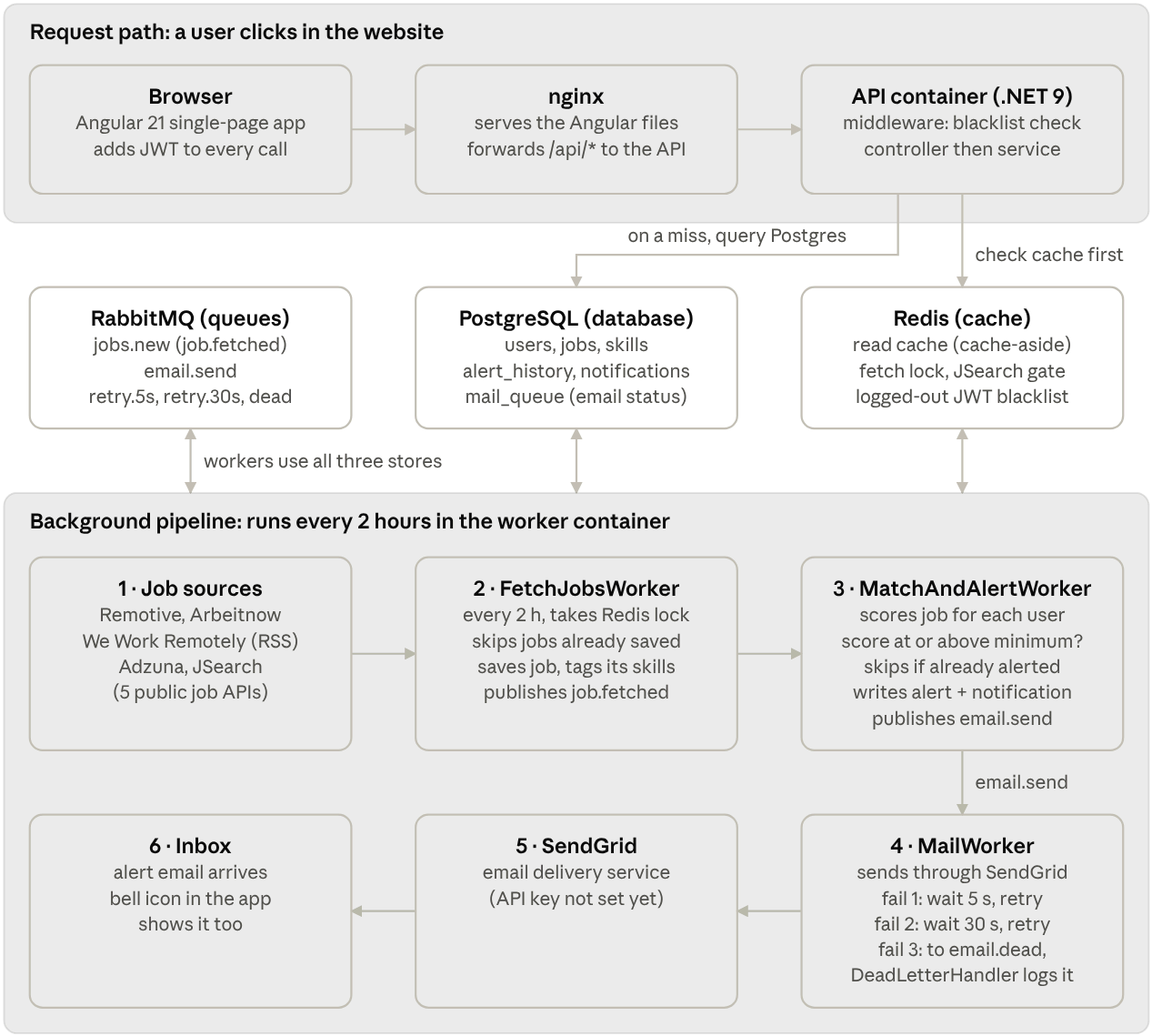
- `src/JobPing.API/Controllers/` — Auth, Jobs, MasterData, Preferences, Admin, Alerts, Notifications
- `src/JobPing.API/Middleware/` — `ExceptionMiddleware` (turns errors into clean JSON), `JwtBlacklistMiddleware` (blocks logged-out tokens)
- `src/JobPing.Infrastructure/Services/` — AuthService, JwtService, JobService, JobFetchService, MatchingService, AlertService, NotificationService, SendGridEmailService, RedisCacheService, SkillMatcher, etc.
- `src/JobPing.Infrastructure/Workers/` — FetchJobsWorker, MatchAndAlertWorker, MailWorker, DeadLetterHandler
- `src/JobPing.Infrastructure/ExternalClients/` — one client per job source (Remotive, Arbeitnow, WWR RSS, Adzuna, JSearch)

- `src/JobPing.Infrastructure/Messaging/RabbitMqTopology.cs` — the single list of all queue/exchange names
- `src/JobPing.Infrastructure/Migrations/` — EF Core database migrations
- `jobping-ui/` — the Angular app (`pages/`, `components/`, `services/`)
- `nginx/`, `Dockerfile`, `docker-compose.yml`, `docs/DEPLOYMENT.md` — packaging and deployment

Every API response has the same shape: `{ success, data, message }` (the `ApiResponseDto<T>` wrapper). The diagram in section 4 shows how these parts talk to each other at runtime.

4. How it works, end to end

There are two separate flows: the **request flow** (a user clicking in the website) and the **background pipeline** (jobs → matches → emails, running on its own every 2 hours).



JobPing end-to-end architecture · request path + 6-step background pipeline

Read it top to bottom: the website (top band) only reads data from the three shared stores; the worker container (bottom band) fills them, moving each new job through steps 1–6. The subsections below explain each part.

4a. Request flow (user clicks something)

1. An Angular service (e.g. `job.service.ts`) makes an HTTP call.
2. The `authInterceptor` adds the JWT token from `localStorage` as `Authorization: Bearer …`.

3. `JwtBlacklistMiddleware` checks Redis: has this token been logged out? If yes, reject.
4. The controller calls a service (e.g. `JobService`).
5. The service checks Redis cache first. On a miss, it queries Postgres, saves the result in Redis with a time limit, and returns it.
6. The result comes back wrapped as `{ success, data, message }`.
7. If any call returns 401 (not logged in), the interceptor clears the tokens and sends the user to `/auth/login`.

4b. Login and tokens

- Passwords are stored as **BCrypt hashes**, never plain text.
- Login returns two tokens: an **access token** (JWT, valid 15 minutes) and a **refresh token** (longer-lived).
- When the access token expires, the app calls `/api/auth/refresh` to get a new pair. The old refresh token is replaced ("rotation").
- Logout calls `/api/auth/revoke`, which puts the token in a Redis **blacklist** until it would have expired anyway.
- Wrong email and wrong password give the **same error** ("Invalid email or password.") so attackers can't discover which emails are registered.
- Admin users have `role_id = 1`; only they can call `/api/admin/fetch-jobs`.

4c. Background pipeline (the heart of the project)

1. **FetchJobsWorker** wakes up every 2 hours (it is just a timer). It calls `JobFetchService.FetchAndStoreJobsAsync`. Admins can trigger the same method on demand.
2. **Lock:** it takes a Redis lock `lock:fetch-jobs` (90 min) so two copies can't fetch at the same time.
3. **Fetch:** it calls all 5 job APIs. If one source fails, it returns an empty list and the others continue.
4. **Dedup + save:** a job is skipped if `(external_id, source)` already exists. New jobs are saved, and `SkillMatcher` reads the title/description to tag which skills the job needs.
5. **Publish:** for each new job it publishes a `job.fetched` message to RabbitMQ.
6. **MatchAndAlertWorker** receives each message. It scores the job against every active user who wants emails. Score = (user's skills that the job needs ÷ total skills the job needs) × 100.

7. If the score is at or above the user's minimum and the user was **not already alerted** for this job, it writes three rows: `alert_history`, `notifications` (the bell icon), and `mail_queue` (status Pending). Then it publishes an `email.send` message.
8. **MailWorker** receives `email.send` and calls SendGrid. On failure it retries with a delay ladder (4d below).

4d. Email retry ladder

If sending fails, the email is tried **3 times in total**:

1. Attempt 1 fails → message goes to `email.retry.5s` → waits 5 seconds → back to `email.send`.
2. Attempt 2 fails → message goes to `email.retry.30s` → waits 30 seconds → back to `email.send`.
3. Attempt 3 fails → `mail_queue.status = Failed`, message goes to `email.dead`, and **DeadLetterHandler** logs it.

The retry queues have no consumer; their only job is to hold a message until its time-to-live (TTL) expires, then RabbitMQ's dead-letter exchange (DLX) sends it back. The attempt count is stored in Postgres (`mail_queue.no_of_attempts`), not in the message. This was tested live and the timings matched exactly.

4e. Matching on the website

`GET /api/jobs/matches` uses `MatchingService.GetTopMatchesAsync`: it loads the user's skills and threshold, scores every active job in memory, keeps those above the threshold, sorts by score then newest, and returns the top 20. The result is cached in Redis for 15 minutes as `user:matches:{userId}`, and that cache is cleared whenever the user changes preferences.

5. Redis and RabbitMQ in plain words

Redis — caching, locking and logout

Redis is an in-memory key-value store: very fast, but data can expire.

Use	Key example	Time limit (TTL)
Cache master data (skills, locations, experience levels)	`skills`, `locations`	24 hours
Cache job list pages	`jobs:list:{MD5 hash of the filters}`	10 minutes
Cache one job's details	job detail key	30 minutes
Cache a user's top matches	`user:matches:{userId}`	15 minutes
Distributed lock for job fetching	`lock:fetch-jobs`	90 minutes
Once-a-day gate for JSearch (protects free quota)	`gate:jsearch-daily`	24 hours
Blacklist of logged-out JWTs	per token	until token expiry
Rate limiting	—	**not built yet**

Cache-aside pattern (used everywhere): check Redis → if missing, read Postgres → save in Redis with TTL → return.

Smart detail to mention: the job list cache is the same for all users. The "saved by me" flag is added *after* reading from cache, so one user's saved jobs never show up for another user.

RabbitMQ — the message post office

RabbitMQ lets one worker hand work to another without waiting. If the receiver is busy or down, the message waits in a queue.

Queue	Who sends	Who receives	Purpose
`jobs.new` (via `job.fetched`)	JobFetchService	MatchAndAlertWorker	"A new job arrived, please match it"

Queue	Who sends	Who receives	Purpose
`email.send`	MatchAndAlertWorker, retry queues	MailWorker	"Please send this email"
`email.retry.5s`	MailWorker	nobody (expires back to `email.send`)	Wait 5 s before attempt 2
`email.retry.30s`	MailWorker	nobody (expires back to `email.send`)	Wait 30 s before attempt 3
`email.dead`	MailWorker	DeadLetterHandler	Final failures, for logging
`notification.create`	nobody yet	nobody yet	Declared for the future; notifications are inserted directly instead

All names live in one file, `RabbitMqTopology.cs`, so the sender and receivers can never disagree on a name.

Why use a queue at all? Fetching jobs should not wait for 239 emails to send. The queue separates the steps, so a slow or broken email service never blocks job fetching, and failed emails can be retried safely.

6. Database — the important tables

Postgres is the single source of truth. The schema is built with **EF Core code-first**: C# classes in `Domain/Entities` plus Fluent API config in `JobPingDbContext.cs` generate the tables through migrations. There are 18 entities; these are the ones you'll be asked about.

Table	What it holds	Important rule
`users`	Account, email, BCrypt password hash, role	Role 1 = Admin

Table	What it holds	Important rule
`user_profiles`	Min match %, email opt-in, etc.	Min match defaults to 50% if no row
`skills`, `locations`, `experiences`	Master lists (seeded on startup)	Cached in Redis 24h
`user_skills`, `user_locations`	Which skills/locations each user picked	Updated by **delta** (add new, remove gone), never delete-all-and-reinsert
`jobs`	Every fetched job	`UNIQUE(external_id, source)` stops duplicates
`job_skills`	Skills detected in each job	Filled by `SkillMatcher`
`saved_jobs` / view history	Favourites and "Viewed" log	—
`alert_history`	Which user was alerted about which job	`UNIQUE(user_id, job_id)` — **never email the same job twice**
`notifications`	Bell-icon messages	Read / unread flag
`mail_queue`	Every email to send, its status and attempt count	Status: Pending / Sent / Failed
`background_tasks`	Status of the fetch job (running, last run)	—

Conventions:

- Table and column names are `snake_case`, applied automatically by a `ToSnakeCase()` helper at the end of `OnModelCreating`.
- All times are UTC (`DateTime.UtcNow`, DB default `NOW()`).
- Development auto-runs migrations on startup; production never does — migrations are run by hand (`dotnet ef database update`).
- Migrations folder is `Infrastructure/Migrations/` (the spec said `Data/Migrations/`; left as-is to avoid breaking migration history).

7. Frontend (Angular 21)

The website is an Angular single-page app in `jobping-ui/`. Every page except `/profile` is built and calls the real API.

Page	Route	What it shows
Landing	`/`	Product intro, how it works, job sources
Login / Register	`/auth/login`, `/auth/register`	Sign-in; 4-step signup wizard
Dashboard	`/dashboard`	Stat cards, top matches, recent jobs, "worker terminal" widget with real last-sync data
Jobs list	`/jobs`	All jobs with filters, pagination, "Best matches" tab, swipe cards on mobile
Job detail	`/jobs/:id`	Full description, match %, apply link
Saved	`/saved`	Jobs the user hearted
Alerts	`/alerts`	Alert settings + history of alerts sent
Preferences	`/preferences`	Skills, locations, threshold; live match-count preview
Profile	`/profile`	**Placeholder ("Coming soon")**

Rules the code follows:

- Standalone components only, `inject()` instead of constructor injection.
- Components never call `HttpClient` directly — every API call goes through a `*.service.ts` (auth, job, master-data, preference, alert, notification, theme).
- Route guards block pages for logged-out users; the `authInterceptor` attaches the token and handles 401s.
- Light and dark theme via CSS variables (`styles/_variables.css`), toggle saved in `localStorage`.
- Responsive: bottom tab bar on mobile, icon-only sidebar on tablet, full sidebar on desktop.

- Icons come from a small home-made `IconComponent` (inline SVGs). The `lucide-angular` library was tried and removed because it added ~14.8 MB to the bundle.

Extra features: `Cmd/Ctrl + K` command palette (navigate, toggle theme, admins can trigger a job fetch), animated match ring on job cards, notification bell with unread count and mark-as-read.

Honesty policy: the UI never shows fake data. If a number isn't available from the API, it shows an empty state instead of a made-up value.

8. Running and deploying

`docker compose up -d` starts the whole system as 6 containers (7 with certbot). It was tested live: 346 real jobs went through the full pipeline inside containers.

Container	What it runs
`api`	The .NET API only (controllers, auth, Swagger) — `RUN_MODE` not set to worker
`worker`	Same image, `RUN_MODE=worker`: only the 4 background workers, no HTTP endpoints
`db`	Postgres 15
`redis`	Redis 7 with a password
`rabbitmq`	RabbitMQ 3 with management UI
`nginx`	Builds the Angular app and serves it; forwards `/api/*` to the `api` container
`certbot`	Gets free HTTPS certificates from Let's Encrypt (for the live server)

Why split api and worker? So the website stays fast even when workers are busy, and each can be restarted or scaled on its own. The split only applies outside Development, so a plain `dotnet run` on your laptop still runs everything in one process.

Running locally without Docker Compose: Postgres runs as a Windows service; start Redis with `docker start redis` and RabbitMQ with `docker start jp-rabbitmq`; then `dotnet run --project src/JobPing.API --launch-profile http` (API on port 5052, Swagger at `/swagger`) and `ng serve` in `jobping-ui` (port 4200).

Secrets: real values go in `.env` (copy from `.env.example`) and `appsettings.Development.json`, both git-ignored. The app refuses to start if `Jwt:SecretKey` is missing.

Live deployment plan (Step 9, not done yet): Oracle Cloud Always Free ARM VM, chosen because it is free forever and big enough to run the whole compose stack. Vercel was rejected (cannot run .NET, Postgres, Redis, RabbitMQ) and Railway was rejected (no real free tier). The step-by-step runbook is `docs/DEPLOYMENT.md`: create VM → open ports 80/443 in both the Oracle firewall and Ubuntu's firewall → install Docker → set `.env` and `DOMAIN` → point DNS → start on HTTP → run migrations through an SSH tunnel → issue the certificate → switch nginx to the HTTPS template → set up auto-renewal. Database and RabbitMQ ports are never opened to the internet.

9. Key design decisions and why

These are the choices people most often ask "why did you do it this way?" about. Each has a one-line reason you can say out loud.

Decision	Why
Fail open if Redis is down (logout blacklist is skipped)	Availability over strict security: a Redis outage shouldn't take the whole API down. Risk is small because access tokens only last 15 minutes.
Fail soft if RabbitMQ is down (publish is logged and dropped)	A missed "new job" message is cheap; an API that won't start is not. Workers retry connecting every 10 s instead of crashing.
Retry ladder with TTL queues + dead-letter exchange	The original spec sent failed emails straight to the dead queue on attempt 1 — no real retries. The ladder gives true 3 attempts with 5 s / 30 s waits and needs no RabbitMQ plugin.
Attempt count stored in Postgres, not in the message	The message is recreated on each retry; the database is the reliable record.

Decision	Why
Notifications inserted directly, not via a queue	It already happens inside the same worker; a queue round-trip adds delay and a failure point for no benefit. Queue is declared in case it's needed later.
Matching done in memory, skill-based only	Simple and fast at the current scale (hundreds of jobs). Would move to SQL if jobs grow to tens of thousands. Location/salary filtering was left out because it wasn't in the spec.
One shared `JobFetchService` for timer and admin button	Both run identical logic; new sources added once show up in both.
JSearch runs once per day (Redis gate)	Every 2 hours would use ~1,800 calls/month against a 500 free limit; daily uses ~150.
Boundary-aware skill matching	Plain `.Contains` matched "Java" inside "JavaScript". Now a match only counts if the characters around it are not letters/digits, so ".NET Core" and "Node.js" still work.
Delta updates for user skills/locations	Only add new and remove removed rows; row IDs stay stable and history isn't wiped.
Same error for wrong email or password	Doesn't reveal which emails have accounts.
Separate `api` and `worker` containers	Matches the spec's design; heavy background work never slows the website.
nginx builds the Angular app (multi-stage Dockerfile)	The spec's plain `nginx:alpine` image would only ever have shown nginx's default page.
HTTP only until a real domain + certificate exist	A config pointing at certificates that don't exist would stop nginx from starting.
Own icon component instead of an icon library	The library added ~14.8 MB; the hand-built one adds a few KB.
No fake data in the UI	Empty states are honest; invented numbers would mislead.

10. Real bugs found by running it — good stories to tell

None of these could be caught by just compiling (`dotnet build` / `ng build`). They were found by actually running the system. In an interview, each one is a strong "tell me about a bug you fixed" answer.

1. **MailWorker silently stuck.** With no SendGrid key, creating the SendGrid client threw an error *before* the try/catch. RabbitMQ's library swallowed it: no ack, no log, no crash. 68 emails sat untouched with 0 attempts. **Fix:** wrapped the whole message handler in an outer try/catch that logs and rejects the message, so config errors are loud.
2. **Wrong container port.** .NET 8+ images listen on port 8080 by default, but everything assumed port 80, so nginx returned 502 errors. **Fix:** set `ASPNETCORE_HTTP_PORTS=80` in the Dockerfile.
3. **Redis password never passed to the app.** The spec turned on a Redis password but never gave it to `api` / `worker`, so every Redis call would fail. **Fix:** added `Redis__Password` to both services.
4. **Orphaned 90-minute lock.** The fetch service wrote to the database *before* the try/finally that releases the Redis lock. When that write failed, the lock stayed held for 90 minutes with nothing running. **Fix:** the try/finally now covers everything after taking the lock.
5. **"Java" matched "JavaScript".** Substring matching tagged every JavaScript job as Java. **Fix:** boundary-aware matching (section 9).
6. **Broken retry design in the spec.** Failed emails dead-lettered on the first attempt, and a 5-minute TTL could drop unsent emails. Fixed in the design *before* coding (section 4d).
7. **Build problems in Docker:** `package-lock.json` out of sync (so `npm ci` failed), production build trying to download Google Fonts during build, and no `environment.prod.ts` (the live site would have called `localhost:5052`). All fixed.
8. **UI bugs:** selected skill chips had invisible text (a CSS layering issue), and toast messages were hard-coded to dark colours in light mode. Both fixed at the root CSS.

11. Where the project stands

Steps 1–8 of the 10-step build plan are done and the backend is verified live; deployment and CI/CD are not started.

Step	What	Status
—	Auth (register, login, refresh, logout, admin role)	Done
1	Master data APIs + Redis cache	Done
2	User preferences API	Done
3	Angular onboarding wired to APIs	Done
4	FetchJobsWorker (now 5 sources)	Done
5	Jobs API + cache	Done
6	Matching engine	Done
7	RabbitMQ + MatchAndAlert + Mail workers	Done, tested live (239 jobs → 62 matches)
—	Alerts + Notifications APIs and UI	Done
8	Docker Compose (6 services)	Done, tested live (346 jobs)
9	Deploy to live server (Oracle Cloud)	Prepared, not done — needs a VM and a domain
10	GitHub Actions CI/CD	Not started

Known gaps — be honest about these if asked:

- **No automated tests** anywhere. Everything was verified by running it manually.
- **No real browser testing** of the frontend yet; only build checks and curl checks.
- **No real emails sent** — SendGrid key is empty, so sends fail (and correctly go through the retry ladder).
- **Adzuna and JSearch** need free API keys before they return jobs.
- **Rate limiting** (Redis) is in the spec but not built.
- **`/profile` page** is a placeholder.
- **Matches API capped at 20**, so the live match count on Preferences undercounts above 20.
- **Swipe-left on mobile** only hides the card for that page load; nothing is saved.
- **Containers have no auto-restart** locally; Redis/RabbitMQ need starting after a reboot.

Git / security housekeeping:

- A placeholder dev JWT secret is in the public repo's first commit. Recommended fix: change (rotate) the secret value rather than rewrite history.
- Local commit `c43c6af` (adds `.gitignore`, stops tracking `bin/obj` and `appsettings.Development.json`) is not pushed yet. Several later changes are also uncommitted.
- Some test data (68 stuck `mail_queue` rows, a test user) is left in the local dev database; safe to wipe.

Next steps, in order:

- [ ] Click through the site in a real browser (light + dark, mobile + desktop)
- [ ] Commit and push pending work; rotate the dev JWT secret
- [ ] Get Adzuna, JSearch and SendGrid keys
- [ ] Create the Oracle VM + domain and follow `docs/DEPLOYMENT.md`
- [ ] Add tests, then GitHub Actions CI/CD
- [ ] Rate limiting and the profile page

12. Likely questions — and how to answer

Q: Walk me through what happens when a new job is posted. Within 2 hours FetchJobsWorker pulls it, checks it isn't a duplicate, saves it, tags its skills and publishes `job.fetched`. MatchAndAlertWorker scores it against every user; users above their threshold who weren't alerted before get a notification and an email row. MailWorker sends the email with up to 3 attempts.

Q: How is the match percentage calculated? (Number of the job's skills the user has ÷ total skills the job needs) × 100, rounded. A job with no detected skills scores 0. Example: job needs C#, Angular, Docker, Azure; user has C#, Angular → 2 ÷ 4 = 50%.

Q: How do you stop sending the same alert twice? The `alert_history` table has a unique key on `(user_id, job_id)`, and the worker checks it before alerting.

Q: How do you stop duplicate jobs? Unique key on `(external_id, source)` in `jobs`, plus a code check before inserting.

Q: What if two servers try to fetch jobs at once? A Redis distributed lock (`lock:fetch-jobs`, 90 min). Only the holder fetches; the lock is always released in a `finally` block.

Q: Why RabbitMQ and not just call the methods directly? It decouples the steps. Fetching never waits for emails, a slow email provider can't block anything, and failed emails retry on a schedule. Messages also survive if a worker restarts.

Q: How do retries work? TTL + dead-letter queues: fail → `email.retry.5s` → back to `email.send`; fail → `email.retry.30s` → back; third fail → `email.dead`. The attempt count lives in Postgres.

Q: How does logout work with JWTs, which can't be "cancelled"? The token is added to a Redis blacklist until its natural expiry; middleware rejects blacklisted tokens. Access tokens last only 15 minutes, with refresh-token rotation.

Q: What happens if Redis or RabbitMQ goes down? The app keeps running. Redis down: the blacklist check is skipped (fail open). RabbitMQ down: publishes are logged and dropped; workers retry connecting every 10 s.

Q: Why cache, and how do you keep it correct? Master data rarely changes (24 h cache); job lists change every 2 h (10 min cache). A user's match cache is deleted whenever they change preferences. The shared job list cache never contains per-user data.

Q: How would it scale? Run more `worker` containers (RabbitMQ shares messages between them; the Redis lock stops double fetching). Move match scoring into SQL once jobs reach tens of thousands. Add rate limiting.

Q: What would you improve? Automated tests and CI/CD, rate limiting, location/salary in matching, a real deployment with HTTPS, and monitoring.

Q: Was it built with AI? Be honest: yes, with an AI coding assistant, working from a detailed spec (`CLAUDE.md`). Then show you understand it — the design decisions (section 9) and the bugs found by running it (section 10) are where you show judgement.

13. Glossary

Term	Meaning in this project
Cache-aside	Check the cache first; if missing, read the DB and store the result in the cache
TTL	Time-to-live: how long a Redis key or RabbitMQ message lives before it expires

Term	Meaning in this project
Distributed lock	A Redis key that only one process can hold, so two workers don't do the same job
Dedup	Removing duplicates (same job twice, same alert twice)
DTO	Data Transfer Object: the shape of data sent to/from the API
DI	Dependency Injection: .NET creates services and hands them to classes that need them (set up in `Program.cs`)
EF Core code-first	Write C# classes; EF Core generates the database tables via migrations
Fluent API	Configuring tables/columns in code (`OnModelCreating`) instead of attributes on classes
Migration	A versioned script that changes the database schema
JWT	JSON Web Token: a signed token proving who you are; sent on each request
Refresh token rotation	Each refresh gives a new refresh token and invalidates the old one
BCrypt	A slow, salted password-hashing method
Middleware	Code that runs on every request before the controller (e.g. blacklist check, error handling)
BackgroundService / Worker	A .NET class that runs on its own in the background (e.g. every 2 hours, or when a message arrives)
Exchange / Queue	RabbitMQ: an exchange routes messages; a queue holds them until a consumer takes them
Ack / Nack	Consumer tells RabbitMQ "done" (ack) or "failed" (nack) for a message
DLX (dead-letter exchange)	Where RabbitMQ sends a message when it expires or is rejected

Term	Meaning in this project
Fail open / fail soft	Keep the app running when a helper service is down, accepting a small risk
`RUN_MODE`	Environment variable that makes the same app image run as API-only or worker-only
Standalone component	Angular component that doesn't need an NgModule
Interceptor	Angular code that modifies every HTTP request/response (adds the token, handles 401)
Guard	Angular code that blocks a route unless the user is logged in
Multi-stage Dockerfile	One stage builds the app (Node), a second, smaller stage runs it (nginx)
Certbot / Let's Encrypt	Free tool and service for HTTPS certificates